COGNITION OS

Mental Health Prediction Platform

End-to-End Technical Architecture & Implementation Guide

Field	Value
Document Version	1.0 — Phase 0 → Phase 2
Last Updated	June 2026
Classification	Confidential — Internal Use
Status	Active Development
Author	CognitionOS Core Team

Table of Contents

Table of Contents 2
1. Executive Overview 5
1.1 Core Differentiators 5
1.2 Build Sequence 5
2. System Architecture 6
2.1 Architecture Pattern 6
2.2 High-Level Component Map 6
2.3 Data Flow (End to End) 6
3. Database Design 8
3.1 Technology Stack 8
3.2 Core Schema — Domain Models 8
3.2.1 users 8
3.2.2 sessions 8
3.2.3 telemetry_events 9
3.2.4 sentiment_scores 9
3.2.5 cognitive_scores 9
3.2.6 consent_records 10
3.2.7 interventions 10
3.3 Indexing Strategy 11
3.4 Migrations 11
4. Backend — FastAPI Service Layer 12
4.1 Technology Choices 12
4.2 API Route Map 12
4.3 Authentication & Authorization 13
4.4 Request Validation Pattern 13
4.5 Background Tasks 13
5. AI / ML Layer 14
5.1 Model Architecture Overview 14
5.2 Sentiment Model — Implementation Detail 14
5.3 TemporalCognitiveRNN 15
5.4 BehavioralDriftDetector 15
5.5 RAG Memory Architecture 15
5.6 LangGraph Multi-Agent Chain 15
5.7 Fine-Tuning Strategy 16
6. Frontend 17

6.1 Technology Stack 17
6.2 Application Structure 17
6.3 Chat Interface Design Principles 17
7. Redis — Caching & Event Bus 19
7.1 Usage Patterns 19
7.2 Redis Configuration 19
8. Security Layer 20
8.1 Data Privacy Architecture 20
8.2 Encryption 20
8.3 API Security 20
8.4 Ethical Governance Controls 21
9. Docker & Containerization 22
9.1 Container Architecture 22
9.2 Docker Compose — Development 22
9.3 GPU Support 22
10. CI/CD Pipeline 23
10.1 Pipeline Overview 23
10.2 Pipeline Stages 23
10.3 Branch Strategy 23
10.4 Model Versioning 23
11. Kubernetes — Phase 5 Production 25
11.1 Do You Need Kubernetes? 25
11.2 Kubernetes Architecture (Phase 5) 25
11.3 Key Manifests 25
12. Monitoring & Observability 26
12.1 Observability Stack 26
12.2 Key Metrics to Track 26
12.3 ML-Specific Monitoring 26
13. Phase-by-Phase Build Plan 28
Phase 1 — Data Foundation (Weeks 1–4) 28
Phase 2 — Real ML Inference (Weeks 5–10) 28
Phase 3 — Fine-tuning & RAG (Weeks 11–18) 28
Phase 4 — Voice Pipeline (Weeks 19–26) 28
Phase 5 — Production Hardening (Weeks 27–34) 28
14. Infrastructure Cost Estimate 30
Development (Phase 1–2) 30

Production (Phase 5 — 1,000 users) 30
15. Technology Decisions Log 31
16. Glossary 32

1. Executive Overview

Cognition OS is an AI-powered mental health prediction and early care management platform. Its core mission is to detect cognitive and emotional drift in individuals before it escalates into a clinical crisis — using behavioral signals, voice input, journaling, and longitudinal memory, without requiring users to fill forms or self-diagnose.

The system is being built model-first: a high-accuracy (90%+) multi-modal understanding layer forms the foundation, on top of which a product experience (chat, streaks, daily tasks, persona-aware responses) is layered.

1.1 Core Differentiators

- No forms — the system understands users through natural conversation and behavior
- Zero PII storage — all personally identifiable information is scrubbed before persistence
- Gender-aware and culturally contextual — adapts to Indian family dynamics, collectivist context
- Proactive, not reactive — the system reaches out; it does not wait for a crisis
- 90%+ prediction accuracy target — multi-modal fusion of voice, text, and behavioral signals
- Ethical-first architecture — consent verification and bias auditing are structural, not added later

1.2 Build Sequence

Phase	Focus
Phase 0 (Done)	Architecture skeleton, mocked services, frontend dashboard
Phase 1 (Current)	Database schema, auth, wire gateway to real services
Phase 2	Real ML inference — sentiment, RNN, drift detection
Phase 3	RAG memory, LangGraph agents, fine-tuned LLM
Phase 4	Voice pipeline (Whisper + Hume), mobile app
Phase 5	Production hardening — Kubernetes, observability, security audit

2. System Architecture

2.1 Architecture Pattern

Cognition OS uses a Service-Oriented Architecture (SOA) with an API Gateway pattern. Each domain (ingestion, analytics, cognitive, agents, memory, governance) lives as an independently deployable service sharing a common data bus (Redis) and persistence layer (PostgreSQL + ChromaDB).

In Phase 1-2, services run as co-located FastAPI routers within a monorepo. In Phase 5, they are extracted into separate containers orchestrated by Kubernetes.

2.2 High-Level Component Map

Layer	Component	Technology
Client	Web Dashboard	Next.js 14, Tailwind CSS
Client	Mobile App (Phase 4)	React Native
Gateway	API Router	FastAPI + Uvicorn
Ingestion	Telemetry Parser	Polars, Pydantic
Cognitive	NLP + Sentiment	SpaCy, HuggingFace Transformers
Analytics	Drift Detection	Scikit-learn, PyTorch LSTM
Agents	Multi-Agent Loop	LangGraph, LangChain
Memory	Vector Store	ChromaDB (dev), Pinecone (prod)
Governance	Ethics Auditor	Custom rule engine
Persistence	Relational DB	PostgreSQL 16 + asyncpg
Cache / Bus	In-memory Store	Redis 7
Observability	Metrics + Tracing	Prometheus, Grafana, MLflow
Infrastructure	Container Orchestration	Docker → Kubernetes (Phase 5)

2.3 Data Flow (End to End)

1. User speaks or types → Frontend captures input → Whisper transcribes voice (Phase 4)
2. Raw input hits POST /api/v1/ingest → Gateway routes to TelemetryIngestor
3. JournalCognitiveParser scrubs PII (SpaCy NER) → anonymized text stored
4. Sentiment pipeline extracts emotional spectrum → scores persisted to PostgreSQL
5. TemporalCognitiveRNN processes sequence of past N sessions → cognitive load score
6. BehavioralDriftDetector compares current session to user baseline → drift flag

7. CognitionAgentOrchestrator (LangGraph) synthesizes all signals → intervention
8. EthicalGovernanceAuditor validates consent and bias before any output is served
9. Response streamed back to client; vector memory updated in ChromaDB/Pinecone

3. Database Design

3.1 Technology Stack

Database	Purpose
PostgreSQL 16	Primary relational store — users, sessions, telemetry, consent
Redis 7	Session cache, rate limiting, pub/sub event bus, hot model results
ChromaDB (dev)	Vector embeddings for RAG memory — local development
Pinecone (prod)	Managed vector DB for production RAG — scalable, low-latency
Alembic	Database migration management
SQLAlchemy 2.0	Async ORM with asyncpg driver

3.2 Core Schema — Domain Models

3.2.1 users

Column	Type	Notes
id	UUID PRIMARY KEY	Generated server-side
created_at	TIMESTAMPTZ	UTC, auto-set
updated_at	TIMESTAMPTZ	Auto-updated trigger
email_hash	VARCHAR(64)	SHA-256 hash — never plaintext
gender	ENUM(m,f,nb,u)	Used for persona adaptation
age_bracket	SMALLINT	Bucketed — not exact DOB
language_pref	VARCHAR(10)	BCP-47 language code
persona_type	VARCHAR(32)	student/professional/parent etc.
consent_version	VARCHAR(16)	Version of consent accepted
tier	ENUM(free,premium)	Subscription tier
is_active	BOOLEAN	Soft delete flag

3.2.2 sessions

Column	Type	Notes
id	UUID PRIMARY KEY	

user_id	UUID FK → users	Indexed
started_at	TIMESTAMPTZ	
ended_at	TIMESTAMPTZ	Nullable — ongoing session
input_modality	ENUM(voice,text,both)	
language_detected	VARCHAR(10)	
raw_input_hash	VARCHAR(64)	Hash only — no raw text stored
session_quality	FLOAT	0.0–1.0 completeness score

3.2.3 telemetry_events

Column	Type	Notes
id	BIGSERIAL PRIMARY KEY	
session_id	UUID FK → sessions	Indexed
event_type	VARCHAR(64)	typing_pause, hesitation, etc.
value	JSONB	Flexible payload
captured_at	TIMESTAMPTZ	
attention_score	FLOAT	Fragmentation metric

3.2.4 sentiment_scores

Column	Type	Notes
id	UUID PRIMARY KEY	
session_id	UUID FK → sessions	
user_id	UUID FK → users	Denormalized for fast query
model_version	VARCHAR(32)	Which model produced this
valence	FLOAT	-1.0 to 1.0
arousal	FLOAT	0.0 to 1.0
dominance	FLOAT	0.0 to 1.0
emotional_label	VARCHAR(32)	Top predicted emotion
confidence	FLOAT	Model confidence
spectrum	JSONB	Full emotion distribution

3.2.5 cognitive_scores

Column	Type	Notes
id	UUID PRIMARY KEY	
user_id	UUID FK → users	
session_id	UUID FK → sessions	
cognitive_load	FLOAT	0.0–1.0 overload indicator
drift_score	FLOAT	Mahalanobis distance from baseline
rnn_confidence	FLOAT	LSTM model confidence
baseline_delta	FLOAT	Change vs 30-day rolling mean
alert_level	ENUM(none,low,med,high,crisis)	
scored_at	TIMESTAMPTZ	

3.2.6 consent_records

Column	Type	Notes
id	UUID PRIMARY KEY	
user_id	UUID FK → users	
consent_type	VARCHAR(64)	data_processing, escalation, etc.
granted	BOOLEAN	
version	VARCHAR(16)	Consent policy version
ip_hash	VARCHAR(64)	For audit trail
recorded_at	TIMESTAMPTZ	

3.2.7 interventions

Column	Type	Notes
id	UUID PRIMARY KEY	
user_id	UUID FK → users	
session_id	UUID FK → sessions	
agent_chain	JSONB	LangGraph execution trace
recommendation	TEXT	Anonymized text output
task_list	JSONB	Daily tasks generated

ethics_passed	BOOLEAN	Governance audit result
delivered_at	TIMESTAMPTZ	

3.3 Indexing Strategy

- users: index on email_hash, persona_type, tier
- sessions: composite index on (user_id, started_at DESC)
- telemetry_events: index on (session_id, captured_at), partition by month
- sentiment_scores: index on (user_id, session_id), composite on (user_id, scored_at DESC)
- cognitive_scores: index on (user_id, alert_level) for fast crisis queries

3.4 Migrations

All schema changes managed through Alembic. Never edit schema manually in production.

```
alembic upgrade head          # apply all pending migrations
alembic revision --autogenerate -m 'add_cognitive_scores'
alembic downgrade -1          # rollback one migration
```

4. Backend — FastAPI Service Layer

4.1 Technology Choices

Technology	Reason
FastAPI	Async-native, automatic OpenAPI docs, Pydantic validation, best Python perf
Uvicorn + Gunicorn	ASGI server — async I/O, multi-worker in production
asyncpg	Fastest PostgreSQL async driver — essential for concurrent session handling
SQLAlchemy 2.0 async	ORM with async session management, Alembic integration
Pydantic v2	Request/response validation, settings management
python-jose	JWT token creation and verification
passlib + bcrypt	Password hashing (admin accounts only — users have OAuth)
httpx	Async HTTP client for internal service calls
celery + redis	Background task queue — model inference jobs, weekly recaps

4.2 API Route Map

Method + Path	Service	Description
POST /auth/register	Auth	Create user, hash email, set consent
POST /auth/login	Auth	Return JWT access + refresh tokens
POST /auth/refresh	Auth	Rotate JWT refresh token
POST /api/v1/ingest	Ingestion	Accept telemetry stream or text input
POST /api/v1/cognitive/analyze	Cognitive	NLP parse + sentiment extraction
GET /api/v1/analytics/score/{user_id}	Analytics	Cognitive load + drift score
POST /api/v1/agents/recommend	Agents	LangGraph intervention generation
GET /api/v1/memory/context/{user_id}	Memory	Retrieve RAG context for user
GET /api/v1/governance/audit/{user_id}	Governance	Consent and bias audit log

GET /api/v1/dashboard/summary/{user_id}	Composite	Full dashboard payload
GET /metrics	Observability	Prometheus scrape endpoint
GET /health	System	Liveness + readiness probe

4.3 Authentication & Authorization

All user-facing endpoints require a valid JWT Bearer token. Tokens are short-lived (15 min access, 7 day refresh). User identity is the UUID — email is never passed in tokens.

JWT PAYLOAD STRUCTURE

```
{ "sub": "<user_uuid>", "tier": "free|premium", "exp": <unix_ts>, "iat": <unix_ts> }
```

Role-based access: regular users access only their own data (enforced at ORM layer via user_id filter). Admin endpoints require an admin role claim and are rate-limited to 10 req/min.

4.4 Request Validation Pattern

Every endpoint uses Pydantic models for input and output. No raw dict passing. All sensitive fields (email, phone) are never accepted after registration — only UUID references.

4.5 Background Tasks

Heavy inference jobs (LSTM scoring, LangGraph agent chains) run as Celery tasks dispatched to a Redis-backed queue. The API returns a task_id immediately; the client polls /tasks/{task_id}/status or receives a WebSocket push on completion.

5. AI / ML Layer

5.1 Model Architecture Overview

Model / Component	Purpose & Approach
HuggingFace SentimentPipeline	Base sentiment: j-hartmann/emotion-english-distilroberta-base — 7 emotion classes
Custom Fine-tuned LLM	Persona-aware response generation — fine-tuned on Indian peer support transcripts
TemporalCognitiveRNN	Bi-directional LSTM — sequence modeling of N past sessions to predict cognitive load
BehavioralDriftDetector	Scikit-learn StandardScaler + Mahalanobis distance from 30-day rolling user baseline
Whisper (Phase 4)	OpenAI Whisper large-v3 — multilingual STT, Hindi/Hinglish/English
Hume AI (Phase 4)	Prosodic emotional tone extraction from voice — arousal, valence, stress markers
SpaCy NER	PII scrubbing — named entities (PERSON, LOC, EMAIL, PHONE) redacted before any storage
RAG Pipeline	LangChain + ChromaDB/Pinecone — retrieves past user context for grounded responses
LangGraph Orchestrator	Multi-step agent chain: Synthesize → Ethics Check → Generate Intervention → Deliver

5.2 Sentiment Model — Implementation Detail

Primary model: j-hartmann/emotion-english-distilroberta-base (distilled RoBERTa, 7 emotion classes: anger, disgust, fear, joy, neutral, sadness, surprise).

Input: PII-scrubbed text from journal/chat session. Output: emotion distribution (softmax probabilities) mapped to valence/arousal/dominance using Russell's circumplex model.

For Hindi/Hinglish: ai4bharat/indic-bert fine-tuned on code-switched sentiment data. Language detection runs first (langdetect library) to route to the correct model.

ACCURACY TARGETS

Metric	Target
English sentiment F1	>= 0.88
Hindi/Hinglish F1	>= 0.82
Emotion classification accuracy	>= 0.85
Latency (P99)	< 250ms

5.3 TemporalCognitiveRNN

Architecture: 2-layer bidirectional LSTM with attention mechanism. Input: sequence of feature vectors from last 30 sessions per user. Features per session: sentiment scores, telemetry attention score, session length, time-of-day, response latency patterns.

Output: cognitive_load (0.0–1.0) and alert_level (none/low/med/high/crisis).

Training data: synthetic sequences generated from clinical literature patterns + anonymized aggregated data. Target accuracy: 90%+ on held-out test set.

TRAINING CONFIGURATION

```
Optimizer: AdamW | LR: 1e-4 | Batch: 32 | Epochs: 50 | Dropout: 0.3
Loss: BCE for alert_level, MSE for cognitive_load | Scheduler:
CosineAnnealingLR
```

5.4 BehavioralDriftDetector

Approach: for each user, maintain a 30-day rolling feature baseline. On each new session, compute normalized Euclidean distance (Mahalanobis approximation) from baseline centroid. Distance > 2.5 sigma triggers a drift flag.

Features: message length distribution, session time patterns, sentiment trend, response speed, vocabulary diversity (type-token ratio).

Cold start: first 7 sessions establish baseline. Drift detection activates from session 8.

5.5 RAG Memory Architecture

Every session produces a memory embedding: a 768-dim vector (using sentence-transformers/all-mpnet-base-v2) encoding anonymized session summary.

Stored in ChromaDB (dev) or Pinecone (prod) with metadata: user_id, session_date, emotional_label, alert_level.

On each new session, top-K (K=5) most relevant past memories are retrieved and injected into the LLM context window as grounding.

MEMORY RETRIEVAL QUERY

```
query_embedding = embed(current_session_summary)
results = vector_store.query(query_embedding, filter={'user_id': uid},
n_results=5)
```

5.6 LangGraph Multi-Agent Chain

The agent orchestrator runs a deterministic graph with the following nodes:

Node	Responsibility
ContextSynthesizer	Merges current session signals + RAG memories + user profile
EthicsGuard	Validates consent, checks for bias, blocks crisis escalation without consent
InterventionGenerator	LLM call — generates personalized task list and response

CrisisRouter	If alert_level=crisis: silently routes to human support queue
ResponseFormatter	Formats output for persona (student/professional/parent tone)

5.7 Fine-Tuning Strategy

The base LLM (Mistral 7B Instruct v0.3 or Llama-3.1-8B-Instruct) is fine-tuned using QLoRA (4-bit quantization with LoRA adapters) on:

- Reddit r/mentalhealth, r/india, r/IndiaSocial anonymized posts
- Kaggle mental health conversation datasets
- Peer support transcript data (with consent)
- Synthetic culturally-aware Indian context conversations

Fine-tuning infrastructure: single A100 80GB GPU, 4–8 hours per run. Served via vLLM for production inference.

QLORA CONFIG

```
r=16, lora_alpha=32, target_modules=['q_proj','v_proj'], lora_dropout=0.05
Quantization: bitsandbytes NF4  |  Batch: 4 grad_accum=8  |  LR: 2e-4
```

6. Frontend

6.1 Technology Stack

Technology	Purpose
Next.js 14 (App Router)	SSR/SSG, API routes, streaming responses
React 18	UI component library
Tailwind CSS v3	Utility-first styling
Shadcn/UI	Accessible component primitives
Zustand	Lightweight global state management
React Query (TanStack)	Server state, caching, background refetch
Framer Motion	Animations — streak counters, emotion cards, transitions
Recharts	Dashboard data visualization — cognitive load history, emotion trends
Socket.io Client	Real-time WebSocket for live session feedback
React Native (Phase 4)	iOS + Android mobile app

6.2 Application Structure

Route	Description
/onboarding	Initial setup — language, persona, consent
/chat	Primary interaction — voice/text input, Buddy response
/dashboard	Cognitive score trends, emotion history, streak tracker
/tasks	Daily task list generated by agent — gamified completion
/recap	Weekly emotional recap card — shareable visual
/settings	Consent management, language, persona, data export
/admin (protected)	Operator dashboard — aggregate anonymized metrics

6.3 Chat Interface Design Principles

- Conversational UI — no forms, no questionnaires visible to user
- Voice-first in Phase 4 — mic always accessible, text fallback
- Streaming responses — token-by-token render via SSE/WebSocket
- Emotional acknowledgment — UI reflects detected emotion subtly (color temperature, card background)
- Streak and task gamification — daily check-in streak, task completion rewards

- Safe escalation UI — if crisis detected, seamless handoff to human support with user consent

7. Redis — Caching & Event Bus

7.1 Usage Patterns

Pattern	Detail
Session Cache	Active session data — TTL 30min, key: session:{session_id}
User Baseline Cache	30-day rolling feature baseline — TTL 24h, key: baseline:{user_id}
Rate Limiting	Sliding window counter per user per endpoint — key: ratelimit:{user_id}:{route}
Task Queue	Celery broker — ML inference jobs, agent chains, email/push notifications
Pub/Sub	Real-time event bus — session.completed, score.updated, crisis.flagged
Hot Model Cache	Last 3 sentiment scores per user — avoid redundant model calls — TTL 10min
JWT Denylist	Revoked tokens stored with TTL matching token expiry

7.2 Redis Configuration

```
maxmemory: 2gb
maxmemory-policy: allkeys-lru
appendonly: yes  (AOF persistence for denylist + task queue durability)
save: 900 1 300 10 60 10000  (RDB snapshots)
```

8. Security Layer

8.1 Data Privacy Architecture

Cognition OS is designed with zero-PII storage as a first principle. This is not a compliance checkbox — it is a core architectural constraint.

PII Category	Handling
Email address	SHA-256 hashed at registration — only hash stored
Name	SpaCy NER scrubs from all text before storage
Location	SpaCy NER scrubs — geo bucket stored (city-level, opt-in only)
Phone number	SpaCy NER scrubs — never stored
Voice audio	Transcribed on-device (Phase 4) or deleted immediately after STT
Raw journal text	Hash stored only — anonymized version in vector store
Device identifiers	Not collected

8.2 Encryption

- All data in transit: TLS 1.3 minimum — enforced at load balancer
- All data at rest: AES-256 encryption — PostgreSQL tablespace encryption + disk encryption
- JWT secrets: stored in environment secrets manager (AWS Secrets Manager / HashiCorp Vault)
- Database credentials: never in code — injected via environment at runtime
- Vector embeddings: encrypted at rest in Pinecone (prod)

8.3 API Security

Control	Implementation
Authentication	JWT Bearer — RS256 asymmetric signing
Rate Limiting	Per-user sliding window via Redis — 60 req/min standard, 10/min admin
Input Validation	Pydantic strict mode — all inputs validated before touching business logic
SQL Injection	SQLAlchemy parameterized queries only — no raw SQL
CORS	Strict allowlist — only known frontend origins
HTTPS enforcement	HSTS header — 1 year max-age

Secrets management	No secrets in code or environment files — Vault/AWS SM only
Dependency scanning	Snyk or Dependabot — weekly automated scan
SAST	Bandit (Python) in CI pipeline — blocks merge on high severity

8.4 Ethical Governance Controls

- Consent must be explicitly granted before any data is processed — no implicit consent
- Bias audits run on model outputs monthly — gender/age group parity checks
- Crisis escalation requires user consent — system never contacts third parties silently
- All governance decisions are logged with immutable timestamps
- Users can request full data export or deletion at any time — DPDP Act (India) compliant

9. Docker & Containerization

9.1 Container Architecture

Container	Image / Base
cognition-gateway	python:3.11-slim — FastAPI + all services
cognition-frontend	node:20-alpine — Next.js production build
cognition-worker	python:3.11-slim + CUDA — Celery workers for ML inference
cognition-postgres	postgres:16-alpine — Primary database
cognition-redis	redis:7-alpine — Cache and message broker
cognition-chromadb	chromadb/chroma:latest — Vector store (dev only)
cognition-prometheus	prom/prometheus:latest — Metrics collector
cognition-grafana	grafana/grafana:latest — Metrics dashboard
cognition-mlflow	ghcr.io/mlflow/mlflow:latest — Experiment tracking

9.2 Docker Compose — Development

Local development uses docker-compose.yml to spin up all services with hot reload. ML models are volume-mounted from a local model cache directory to avoid re-downloading.

```
docker compose up --build          # start all services
docker compose up gateway frontend  # start subset
docker compose --profile gpu up     # include CUDA worker
```

9.3 GPU Support

ML inference workers (Celery) require NVIDIA GPU access in production. Docker Compose GPU profile mounts the NVIDIA runtime. In Phase 2, a single T4/A10 GPU instance handles all inference. Phase 5 separates inference into a dedicated GPU node pool in Kubernetes.

10. CI/CD Pipeline

10.1 Pipeline Overview

CI/CD is implemented in GitHub Actions with the following stages: Lint → Test → Security Scan → Build → Push → Deploy.

10.2 Pipeline Stages

Stage	Tools & Actions
Lint	ruff (Python), eslint + prettier (TypeScript) — fail fast on style violations
Unit Tests	pytest with asyncio support — minimum 80% coverage gate
Integration Tests	pytest + testcontainers — spin up real Postgres/Redis in CI
Security Scan	Bandit (SAST Python), npm audit, Snyk container scan
Build Images	Docker buildx — multi-platform (amd64 + arm64) — layer caching
Push Registry	GitHub Container Registry (ghcr.io) — tagged with commit SHA
Deploy Staging	Render.com or Railway — auto-deploy on merge to develop branch
Smoke Tests	Playwright E2E — 5 critical user paths validated post-deploy
Deploy Production	Manual approval gate — deploy on merge to main
Notify	Slack webhook — pass/fail notification with deploy URL

10.3 Branch Strategy

Branch	Purpose
main	Production — protected, requires 2 approvals + CI pass
develop	Integration branch — deploys to staging automatically
feature/*	Feature branches — merge to develop via PR
hotfix/*	Emergency production fixes — merge to main + develop
model/*	ML experiment branches — tracked in MLflow, not deployed automatically

10.4 Model Versioning

ML models are versioned separately from application code. Model artifacts are stored in S3/GCS and tracked in MLflow. The gateway loads models by version tag from the model registry — promoting a new model version does not require a code deploy.

11. Kubernetes — Phase 5 Production

11.1 Do You Need Kubernetes?

Not yet. In Phase 1–3, Docker Compose on a single VM (4 vCPU, 16GB RAM, T4 GPU) is sufficient and far simpler to operate. Kubernetes becomes necessary when:

- Traffic exceeds ~5,000 concurrent users
- ML inference latency requires dedicated GPU node pools
- You need zero-downtime rolling deployments for model updates
- Multi-region deployment is required

Recommendation: Build Kubernetes manifests in Phase 3 but do not operate them until Phase 5.

11.2 Kubernetes Architecture (Phase 5)

Resource	Configuration
Cluster	GKE Autopilot or EKS — managed control plane
CPU Node Pool	n2-standard-4 × 3 nodes — gateway, frontend, workers
GPU Node Pool	n1-standard-8 + T4 × 2 nodes — ML inference only, scale to zero
Ingress	NGINX Ingress Controller + cert-manager (Let's Encrypt TLS)
Service Mesh	Istio (optional Phase 6) — mTLS between services, traffic policy
Storage	Persistent Volume Claims — PostgreSQL, ChromaDB, MLflow artifacts
Secrets	External Secrets Operator → AWS Secrets Manager / GCP Secret Manager
Autoscaling	HPA on CPU/memory for gateway; KEDA on queue depth for ML workers
Observability	Prometheus Operator + Grafana stack — built into cluster

11.3 Key Manifests

Each service requires: Deployment, Service, HorizontalPodAutoscaler, PodDisruptionBudget, NetworkPolicy. The gateway and frontend have separate Ingress rules. The ML worker Deployment uses nodeSelector to target the GPU node pool.

12. Monitoring & Observability

12.1 Observability Stack

Tool	Role
Prometheus	Metrics scraping — FastAPI /metrics endpoint, Redis, PostgreSQL exporters
Grafana	Dashboards — system health, ML model latency, user session volume
MLflow	ML experiment tracking — model versions, training metrics, artifact storage
Weights & Biases	Training run tracking, hyperparameter sweep visualization
Sentry	Application error tracking — Python backend + Next.js frontend
Structlog	Structured JSON logging from all Python services
OpenTelemetry	Distributed tracing — trace a request end-to-end across services
Alertmanager	Prometheus alerting — PagerDuty/Slack on SLO breach

12.2 Key Metrics to Track

Metric	Alert Threshold
API P99 latency	> 500ms triggers warning, > 2s triggers alert
ML inference P99 latency	> 300ms triggers warning
Model prediction drift	Significant distribution shift triggers retraining job
Crisis alert rate	Spike > 2x baseline triggers on-call page
DB connection pool exhaustion	> 80% pool used triggers scale-up
Celery queue depth	> 100 pending jobs triggers worker scale-out
Sentiment model confidence	Mean confidence < 0.6 triggers model review

12.3 ML-Specific Monitoring

Data drift detection: weekly automated job compares current week's input feature distribution against training distribution using Population Stability Index (PSI). PSI > 0.2 triggers a model review.

Model performance tracking: all predictions logged with ground truth labels collected from user feedback (implicit: escalation accepted/declined; explicit: weekly reflection prompt). F1 score computed monthly.

13. Phase-by-Phase Build Plan

Phase 1 — Data Foundation (Weeks 1–4)

- Define and migrate full SQLAlchemy schema (all 7 domain models)
- Implement JWT authentication — register, login, refresh, revoke
- Wire FastAPI gateway to real service classes (remove all mocked responses)
- Replace mocked sentiment extraction with real HuggingFace pipeline
- Set up Alembic migrations and seed scripts
- Basic test suite — 80% coverage on auth and ingestion paths

Phase 2 — Real ML Inference (Weeks 5–10)

- Train TemporalCognitiveRNN on synthetic + real data — hit 90%+ accuracy target
- Implement BehavioralDriftDetector with real user baseline accumulation
- Connect ChromaDB vector store — real embedding storage and retrieval
- Wire LangGraph agent chain end to end — real LLM calls (GPT-4o or local Mistral)
- Implement EthicalGovernanceAuditor consent checks in production path
- MLflow experiment tracking for all model training runs

Phase 3 — Fine-tuning & RAG (Weeks 11–18)

- Collect and prepare fine-tuning dataset (Reddit, Kaggle, synthetic)
- Fine-tune base LLM with QLoRA — Hindi/Hinglish + English
- Migrate vector store to Pinecone for production-grade retrieval
- Implement longitudinal memory — user context persists across sessions meaningfully
- Build admin dashboard — anonymized aggregate metrics for internal use
- Gender-aware and persona-aware response routing

Phase 4 — Voice Pipeline (Weeks 19–26)

- Integrate OpenAI Whisper large-v3 for multilingual transcription
- Integrate Hume AI for prosodic emotion extraction
- Build React Native mobile app with voice-first interaction
- On-device STT for sensitive audio (privacy-first) — WhisperKit (iOS)
- Proactive push notification system — Buddy reaches out

Phase 5 — Production Hardening (Weeks 27–34)

- Kubernetes cluster setup — GKE or EKS
- Full observability stack — Prometheus, Grafana, OpenTelemetry tracing
- Security audit — penetration test, SAST/DAST, DPDP compliance review
- Load testing — k6 or Locust — validate 5,000 concurrent users
- Disaster recovery runbook — RTO < 1h, RPO < 15min

- Beta launch with 1,000 users — 1 college, 1 city

14. Infrastructure Cost Estimate

Development (Phase 1–2)

Resource	Estimated Monthly Cost (USD)
Single VM — 4 vCPU, 16GB RAM (GCP e2-standard-4)	~$130/mo
T4 GPU instance (spot/preemptible for training)	~$200/mo
PostgreSQL managed (Cloud SQL small)	~$50/mo
Redis managed (Memorystore basic)	~$35/mo
Object storage S3/GCS (model artifacts)	~$20/mo
Total Phase 1–2	~$435/mo

Production (Phase 5 — 1,000 users)

Resource	Estimated Monthly Cost (USD)
GKE Autopilot cluster (3 CPU nodes)	~$300/mo
GPU node pool (T4 × 1, preemptible)	~$150/mo
Cloud SQL PostgreSQL — HA	~$150/mo
Redis Memorystore — standard tier	~$80/mo
Pinecone (pod-based, s1 × 1)	~$70/mo
CDN + Load balancer	~$50/mo
Monitoring stack (managed Grafana)	~$50/mo
Total Phase 5 (1k users)	~$850/mo

15. Technology Decisions Log

Key architectural decisions and the reasoning behind them, for future reference.

Decision	Chosen	Rejected / Why
Primary language	Python (FastAPI)	Node.js — Python ML ecosystem is non-negotiable
ORM	SQLAlchemy 2.0 async	Django ORM — too opinionated, no async support
Vector DB (dev)	ChromaDB	Weaviate — too heavy for local dev
Vector DB (prod)	Pinecone	Self-hosted Weaviate — operational burden too high
LLM serving	vLLM	TGI — vLLM has better throughput for batched inference
Agent framework	LangGraph	AutoGen — LangGraph has explicit state control
Fine-tuning method	QLoRA	Full fine-tune — hardware cost and time prohibitive
Mobile (Phase 4)	React Native	Flutter — team JS/TS familiarity
Container orchestration	Kubernetes (Ph5)	Nomad — K8s ecosystem maturity
Monitoring	Prometheus + Grafana	Datadog — cost at scale

16. Glossary

Term	Definition
Cognitive Load	Measure of mental effort — 0.0 (minimal) to 1.0 (overloaded)
Behavioral Drift	Deviation from a user's established behavioral baseline
RAG	Retrieval-Augmented Generation — grounds LLM responses in retrieved memory
LangGraph	Framework for building stateful multi-step AI agent workflows
QLoRA	Quantized Low-Rank Adaptation — efficient LLM fine-tuning with less GPU RAM
PII	Personally Identifiable Information — names, emails, phone numbers, locations
Mahalanobis Distance	Statistical distance accounting for feature correlations — used in drift detection
PSI	Population Stability Index — measures distribution shift between two datasets
DPDP Act	Digital Personal Data Protection Act 2023 — India's data privacy law
vLLM	High-throughput LLM inference server with PagedAttention
HPA	Horizontal Pod Autoscaler — Kubernetes auto-scaling based on metrics
KEDA	Kubernetes Event-Driven Autoscaling — scales on queue depth, custom metrics
SOA	Service-Oriented Architecture — services communicate via well-defined APIs
SAST	Static Application Security Testing — code analysis for vulnerabilities

— End of Document —

Cognition OS | Confidential | Version 1.0 | June 2026